



Fetal health risk prediction using ensemble-based machine learning approaches

Subhash Mondal^{1,2} · Ranjan Maity¹ · Amitava Nag¹ · Soumadip Ghosh³

Received: 23 October 2022 / Revised: 21 March 2025 / Accepted: 4 April 2025 /

Published online: 10 May 2025

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract

In 2017, WHO reported a rise in neonatal and maternal deaths in developing countries. Congenital impairments, bad maternal lifestyle, hunger, resource shortages, and absence of fetal health check-ups cause newborn, maternal, and postnatal mortality. To tackle the above concerns in a cost-effective and efficient approach, we tried to predict the risks of fetal health based on machine learning (ML) techniques and categorize them into three classes: pathological, suspect, and normal. Consequently, we can make informed decisions if the fetus exhibits critical issues. The study uses a cardiotocography dataset with 2126 cases, 21 independent features, and a multiclass target attribute in a three-step process to predict the health risk of the fetus. The first stage deploys sixteen ML models following the necessary data preprocessing, both with and without sampling methods. In the second stage, we selected eleven of the best performing classifiers to perform hyperparameter tuning using RandomizedSearchCV and GridSearchCV. Finally, we proposed a fine-tuned ensemble learning (EL) approach based on stacking, voting, and bagging for the precise fetal health status prediction. These models allow early diagnosis of fetal health conditions to be identified and treated with minimal time and expense. It was observed that the proposed stacking, bagging, and voting methods gave ten-fold mean accuracies of 99.60%, 99.72%, and 99.78%, respectively. Using ensemble classifiers can improve and eliminate the weakness of one individual classifier, yielding superior performance for the overall model. This study readily offers healthcare providers useful interpretations to aid in their decision-making.

✉ Amitava Nag
amitava.nag@cit.ac.in

Subhash Mondal
ph22cse1001@cit.ac.in

Ranjan Maity
r.maity@cit.ac.in

Soumadip Ghosh
soumadip.ghosh@gmail.com

¹ Computer Science and Engineering, Central Institute of Technology Kokrajhar, Kokrajhar, Assam, India

² Computer Science and Engineering (AI&ML), Dayananda Sagar University, Bengaluru, Karnataka, India

³ Computer Science and Engineering, Future Institute of Technology, Kolkata, West Bengal, India

Keywords Fetal health · Hyperparameter tuning · Cardiotocography (CTG) · Fetal monitoring · Machine learning (ML) · Ensemble Learning

1 Introduction

Fetal health anomaly is a global concern in underdeveloped and developing nations. A fetus is an unborn offspring that has attained the basic structure and would further grow inside the mother's womb. The fetal period begins eight weeks after conception and ends at the time of birth. During this time, the significant development of body structures is observed. During this period, it is required to take care of both maternal and fetal health, keenly observing for any problems. It is necessary for the mother to have a healthy diet and lifestyle during pregnancy. Congenital disabilities, malnutrition, scarcity of the right resources in low-resource countries, and lack of fetal health check-ups during pregnancy are major contributing factors to neonatal deaths, stillbirths, and under the age of five-years deaths. The total number of child deaths under the age of 5 accounts for almost 8 million every year [1]. The report [2] suggests that there were 2.4 million deaths during the first month of neonatal life in 2019 and approximately 6700 newborn deaths every day, the majority of which could have been averted. Abnormal fetal growth presents considerable hazards to maternal health and death, with an estimated 810 women dying daily during pregnancy and childbirth, as reported in 2017 by WHO [3], with 94% of these fatalities transpired in low and lower-middle-income countries. The World Health Organization prioritizes maternal health. Additionally, in accordance with the additional objective of Sustainable Development Goal 3 (SDG 3.1), it is aimed that no nation should exhibit a maternal death ratio surpassing 140 per 100,000 live births by 2030 [4]. To attain this objective, high-quality data, particularly on causes of mortality, is essential for formulating treatments and mitigating the worldwide burden of maternal mortality [5].

The maternal mortality ratio is elevated in impoverished nations, underscoring the imperative for fetal health surveillance. Medical professionals advocate for routine examinations to evaluate fetal condition, with Cardiotocography (CTG) being a prevalent method for screening both fetal and maternal health [6]. CTG persistently monitors the fetal heart rate (FHR) and uterine contractions (UC) using an ultrasound transducer positioned on the mother's abdomen [7]. The CTG data comprises 21 attributes utilized by obstetricians to assess fetal health status, facilitating early identification and intervention of fetal distress prior to its escalation. Processing CTG data is laborious and susceptible to errors. Artificial Intelligence (AI) facilitates the analysis of CTG data, yielding a more rapid and comprehensive report compared to human interpretation. The advent of artificial intelligence has enabled more progress in prenatal health monitoring [8] [9]. However, accurately predicting prenatal health complications remains a considerable challenge in the field of machine learning [10].

Recently, the increase in the amount of stored data in hospitals and healthcare units has promoted the potential use of AI in healthcare. Identifying patterns in data by training these massive datasets using machine learning (ML) models helps early diagnosis of diseases. Sometimes these diagnoses are almost instantaneous and provide better accuracy than medical diagnoses, which helps save time and capital. In the case of fetal health diagnosis, early prediction of any fetal anomaly can prevent most neonatal deaths. The classification of fetal health is crucial for ensuring precise and successful prenatal care, as well as for the early identification of potential issues. Conventional approaches may occasionally be susceptible to errors and inconsistencies; therefore, it is imperative to utilize developing tools such as

machine learning and deep learning techniques. The manual processing of CTG data is laborious and inconsistent. The existing techniques are insufficient for fetal health classification. To improve diagnostic accuracy, powerful machine learning techniques can be employed for precise prediction of fetal health. To attain superior outcomes, we recommend a refined ensemble learning (EL) approach. We implement diverse preprocessing approaches, equilibrate the dataset through sophisticated sampling methods, normalize the dataset, data outlier detection, and identify the most significant characteristics from the fetal CTG data. The processed dataset is provided as input from conventional machine learning methods and the ensemble learning models for the precise prediction outcomes.

In this study we employed the UCI CTG dataset containing 2126 patient records of 21 features extracted from the Cardiotocogram assessment and graded by the specialist obstetricians into pathological, suspect, and normal classes. Cases of suspects and pathological fetuses, if treated earlier, can save many neonates. For the diagnosis of fetal health risk, the different conventional machine learning classifiers, namely LGBM, HGB, XGB, AB, CB, GB, RF, DT, SVM, LR, GNB, BNB, and KNN, MLP, LDA, and ETC, are considered to train the prediction model initially. The acronyms of every algorithm used in this study are listed in Table 1. Through the exploratory data analysis (EDA), the shortfalls of raw data are addressed by applying the different preprocessing techniques before the model training. Each model hyperparameter is tuned to find out the optimized parameter value by applying the RandomizedSearchCV (RSCV) and GridSearchCV (GSCV), after that the top-performing tuned models from both the cases are considered and decided to employ ensemble learning techniques to classify fetal health into the above-mentioned classes. The suggested method shows considerable advancements in forecasting fetal health issues through machine learning techniques, highlighting the potential of artificial intelligence in enhancing maternal and fetal health outcomes.

This research aims to automate the classification of fetal health to achieve a prompt and precise diagnosis of fetal and maternal health. Given that CTG anomalies constitute critical medical crises, diagnosis and interpretation necessitate utmost precision. Achieving a high level of accuracy in automating the fetal health classification process necessitates a balanced dataset and robust prediction models. The suggested methodology would rectify the dataset imbalance and employ ensemble classifiers to categorize the CTG records into healthy, suspected, and abnormal classifications. The paper seeks to emphasize the enhanced performance of ensemble classifiers and the resultant improved accuracy attained by the

Table 1 Learning models name and their acronyms

Techniques	Acronym	Techniques	Acronym
Hist Gradient Boost	HGB	Bernoulli Naïve Bayes	BNB
Random Forest	RF	Support Vector Machine	SVM
CatBoost	CB	K-Nearest Neighbor	KNN
Xtreme Gradient Boost	XGB	Logistic Regression	LR
Gradient Boost	GB	Gaussian Naïve Bayes	GNB
Light Gradient Boost Machine	LGBM	AdaBoost	AB
Decision Tree (Classification and Regression Trees (CART))	DT	Multi-Layer Perceptron	MLP
Linear Discriminant Analysis	LDA	Extra Trees Classifier	ETC

application of the soft voting classifier, which computes the average result of all ensemble classifiers. The suggested model will facilitate prompt diagnosis, decrease fetal mortality, and birth difficulties, and enable medical practitioners to include the CTG automation process into the physical examination, thereby enhancing the accuracy and speed of the diagnostic procedure.

We have organized the document as follows: Sect. 2 examines pertinent literature in the domain and delineates the research gaps that our study addresses. Section 3 delineates the materials and methodologies employed in our research, encompassing data sources, study design, data preprocessing, and statistical analysis. Section 4 delineates the experimental findings of the investigation of different types of prediction models with clarity and precision. Section 5 provides an extensive analysis of the results, encompassing a critical assessment of the study. This study concludes in Sect. 6 with its implications, challenges, and limitations for the field and recommendations for future research.

2 Related literatures

The classification of fetal health is crucial for the early identification of potential risk factors related to fetal conditions that can be assessed during routine examinations. During gestation, physicians monitor fetal movements, heart rate, uterine contractions, and growth through numerous methods, including blood tests and ultrasounds. Upon identifying potential risk factors in pregnant women, physicians should promptly implement care to prevent miscarriage. Artificial intelligence-driven CTG image analysis indicates that healthcare practitioners can readily make decisions. Numerous academics have developed various machine and deep learning algorithms to assess fetal condition. The manual processing of CTG data is often susceptible to errors and is time-consuming. These models evaluate the CTG data and assist healthcare practitioners in making prompt decisions. This section considered mainly research articles related to fetal health condition classification using machine learning-based state-of-the-art approaches published in literature. After rigorous searching among various research publication portals, we shortlisted the below mentioned literature for the study of related work.

The authors Hakan Sahin et al. [11] intended to assess the performance of eight different machine learning algorithms like SVM, ANN, KNN, CART, RF, LR, Radial Basis Function Network (RBFN), and C4.5 over the dataset of 2126 instances and 21 features that was obtained using CTG. The tenfold cross-validation was applied to analyze the classification model performance. They also used the WEKA workbench to extract the best parameters. The end results showed that RF performed significantly to classify normal, suspect, and pathological in CTG data. The accuracy, recall, and specificity of the model were recorded as 99.18%, 94.18%, and 99.74%, respectively. In [12], the main objective of the author was to measure the performances of ensemble ML algorithms on the CTG data of antepartum to monitor the fetal heart rate for anomalies. The dataset had 21 independent features and one target feature of 2126 instances. They used eight different algorithmic models to classify the fetal heart rate. Algorithms incorporated in their study were ANN, RF, SVM, KNN, CART, REP Tree, C4.5, and RF. They claimed the result of RF with an accuracy of 98.96%. Further use of bagging ensemble methods increased the performance of RF to an accuracy of 99.02%. In [13], the authors extracted an online available public dataset wherein the data was classified using CTG tests. The dataset was classified based on the fetus's heart rate into three classes. They applied various preprocessing techniques like removing outliers, data scaling and data

balancing. Authors were trained the dataset using various algorithms like XGB, LGBM, RF, SVM, KNN, ANN, and LSTM in three scenarios. They proposed three scenarios by considering different preprocessing methods of which LGBM performed well in all of them with an accuracy of 99%. In [14], the authors used the most extensively used dataset obtained from the UCI repository to determine fetal health using machine learning. They applied twelve algorithms like GB, CB, LGBM, XGB, Cascade Forest classifier (CFC), RF, ETC, DT, LR, KNN, LDA, and AB for classification purposes. Further, they incorporated stacking and soft voting into the top four models. Among these algorithms, the authors claimed that voting method performed best with an accuracy of 95.9%.

Also, the authors Akhan Akbulut et al. [15] intended to help physicians and families to predict fetal inborn irregularities during pregnancy using AI through an Android application. The purpose of this e-health app was to predict the fetal health of those pregnant mothers who were unable to see a physician in a short period of time. They incorporated nine different binary machine learning models, namely, Locally deep SVM, LR, Average Perceptron, Bayes Point Machine, Decision Forest, Decision Jungle, NN, Boosted DT, and SVM for classification. After applying all these algorithms, the highest prediction accuracy was obtained at 89.5% using the DT model on their own clinical dataset. They also used their model on sixteen real-life users which fetched an accuracy of 87.5%. In [16], the authors obtained the dataset from the UCI repository wherein the data was classified by professional obstetricians into three classes of a fetus using CTG tests. The authors intended to lower the risk of under-five mortality by building an ML model which will be easily accessible and can be used to early predicts fetal health in a low-resource setting or in a low-income country. So, if any fetal anomaly is observed the patient can be further referred to take medical help. For the classification model, various ML algorithms like MLP, SVM Linear Kernel, XGB, DT, RF, LR, SVM RBF kernel, AB, GNB, and KNN were incorporated. Among all these algorithms XGB technique had the highest overall accuracy of 93%. Astik Kumar Pradhan et al. [17] used the most used UCI machine learning CTG dataset. The dataset contained 2126 instances and a total of 22 features in which 21 were independent variables and 1 was the target. They used normalization and scaling to preprocess data. The authors used five-fold cross-validation with a train and test data ratio of 0.70:0.30 to train the models using algorithms like LR, KNN, GBM, and RF. It was observed that RF performed significantly better with the CTG dataset and gave an accuracy of 93%. The authors in [18] used the CTG dataset categories into three classes after analyzing the uterine contraction and fetal heart rate namely pathological, suspect, and normal. They used SMOTE to balance the dataset by oversampling the minority class and eliminated outliers during preprocessing. The training process was incorporated algorithms like LGBM, GBM, RF, ETC, and DT wherein they observed that LGBM gave the best test accuracy of 94.5% before optimization using AutoML. They further optimized their model using hyperparameter tuning which resulted in an increase in the accuracy of LGBM to 95.61%. The model prediction was interpreted by using the XAI tools SHAP. In [19], the author's primary intention was to lower the child and the mother's mortality rate during live birth using AI by early classification of fetal heart rate anomalies. They extracted the UCI CTG dataset that had 2126 instances in total and 21 independent variables and one outcome variable. Seven supervised algorithms were applied linear SVM, AB, DT, KNN, Stacking, Bagging, and NB. The authors observed that AB with sub-model RF performed well after the incorporation of tenfold cross-validation with an accuracy of 94.7%. In [20], the authors worked on a CTG dataset extracted from the University of Porto. The dataset had 1831 instances and 21 attributes. The paper was mainly focused on the implication of AI to reduce the mortality rate of the fetus and mother in underdeveloped and developing nations through early prognosis of fetal health. Algorithms like KNN, MLP, SVM, and RF

were incorporated to train and test the models against the CTG dataset for the classification of fetal health. Further regression analysis and correlational analysis were used to monitor the influence of each independent feature on the target variable wherein it was observed that fetal heartbeat deceleration was a significant feature for the classification of fetal health. The authors used various metrics to judge the overall performance of the algorithms like accuracy, recall, F1-score, precision, and support. It was observed that RF performed best among all the models with an accuracy of 94.5% and SVM gave the second-best accuracy of 93%. In [21], the authors proposed an Enhanced Binary Bat algorithm (EBBA) for the selection of important features from CTG dataset to classify the fetus state. Also, applied other evolutionary algorithms like genetic algorithm, quantum Grey Wolf Optimization (qGWO) and finally classify with 11 features using the RF, DT, KNN models. The combine EBBA and RF achieved an accuracy of 96.21% on reduced features.

Moreover, the recent advancement of fetal health classification is observed in a few articles, like in [22], which proposed an ensemble voting method using the five classifiers like CB, AB, XGB, LGBM, and GB for the fetus health classification. They used the CTG dataset with 2126 instances and applied preprocessing with and without sampling and deployed models two times. They observed the accuracy of 99% for soft voting and XGB, CB, and LGBM alone. In study [23], an ensemble voting method was also proposed by considering the LR, RF, GB, and XGB on the UCI CTG dataset. The combined voting of XGB, RF, and GB achieved an accuracy of 99.5% after feature extraction using the polynomial expansion. The authors [24] proposed a deep neural network (DNN) approach on the UCI CTG dataset along with the baseline machine learning models like LR, KNN, SVM, NB, RF, and GB for the fetal health analysis. They obtained remarkable accuracy of 99% compared to other models and indicates the potential use of deep learning method for the assessment of fetal health. Also, using SHAP the feature importance was explained of the prediction model. Also, the authors in [25] proposed an ensemble learning based on dimension reduction technique using the PCA and autoencoders for the fetal health classification on CTG dataset. After the feature extraction, the performance of machine learning models like LR, SVM (RBF and Linear), KNN, NB, DT, RF, AB, GB, XGB, and LGBM was analyzed concerning to PCA and Autoencoder. It was noted that the XGB and GB achieved the accuracy of 99% with cross-validation and applied data preprocessing.

Finally, a few more articles focus on the fetal health classification in recent years. The authors in [26] proposed a novel ensemble method combined with tuned Support Vector Machine and Extra Trees (ETSE) for predicting the fetal health. After applying data preprocessing, seven machine learning models, SVM, LGBM, XGB, KNN, DT, RF, and ETC, were implemented on the CTG dataset. The hyperparameters were tuned by GridSearchCV, and after that, a voting ensemble of different combinations was tested, and the ETSE achieved the highest accuracy of 99.66 among other types. Also, in article [27], a proposed neural network-based Dynamic Multi-Layer Perceptron (DMLP) model to classify the fetal health on CTG dataset. The deployed model performance was compared with the base models like RF, KNN, XGB, DT, and LR results after applying the preprocessing like feature scaling, balancing with SMOTE, selection of features using ANOVA. The highest accuracy was achieved by DLMP model of 97% compared to traditional models. In article [28] proposed a federated machine learning-based fetal health classification model on bio-signal clinical CTG dataset using KNN model. The combined model of FML with KNN achieved an accuracy of 99.06% outperformed base KNN model result. At last, in article [29] proposed a boosting-based fetal health classification model based on tuned XGB considering the CTG dataset. They applied the query function for the feature selection and used grid search cv for parameter tuning.

Finally, after the diversity exploiting and uncertainty criteria applied on active learning (AL) method on the XGB model achieved an accuracy of 99.38%.

It is observed from the above-mentioned literature survey that there is a huge scope to explore the prediction model for determining the fetal health conditions. The recent classification methods have major challenges due to the quality and the variety of input data for monitoring fetal health. Due to the different sources of data, such as biochemical factors, ultrasound images, and fetal heart rate monitors, they can affect the reliability and the accuracy of prediction models. Moreover, these models can face challenges when addressing unbalanced datasets, when instances of specific fetal health conditions are significantly rarer than others. Also, due to the diversified machine learning classification models, it is a very challenging task to select the proper classification model and the well-mannered approaches. Among the various machine and deep learning approaches, it is necessary to address their significant insights and choose which one should be appropriate for the fetal health classifications. Moreover, current proposals may not sufficiently include the ever-changing and complex nature of fetal health, in which several parameters that would associate with one another affect the final classification decision depending on approaches. To tackle these issues, it is necessary to create classification models that are robust to deal with imbalanced datasets, parameters of task-specific models, the right choice of approach, and can deal with the inconsistency during the classification.

Considering the research gap and challenges, we have summarized the primary contributions that are novel in nature as follows:

- (a) Initially, we have selected sixteen diversified machine learning classification models and deployed them with the ten-fold cross-validation method to prevent overfitting and improve generalizability.
- (b) This study examined the significant use of data preprocessing on model training combined with feature scaling, outlier detection, and the effect of advanced sampling methods Synthetic Minority Over-sampling Technique (SMOTE) and SMOTE-Edited Nearest Neighbor (SMOTEENN) to overcome the target class data imbalance issue.
- (c) We employed hyperparameter tuning methods like GridSearchCV and Randomized-SearchCV to optimize the hyperparameters of superior models systematically to combat overfitting and ensure improved performance.
- (d) This study is the first attempt to perform a complete analysis of ensemble learning (EL) approaches for fetal health classification. We employed voting, bagging, and stacking ensembles with all possible combinations of the best performing tuned models and compared their performance by considering different evaluation metrics.
- (e) The stacking model improves its cross-validation mean accuracy to 99.53%. Also, the soft voting ensemble with all non-boosting tuned models performed better, with a mean accuracy of 99.78%, a lower standard deviation, and near 1.00 recall values for the fetal health classification model.

3 Methodology and measurement: dataset and preprocessing

This proposed section includes a few stages as illustrated in Fig. 1, like dataset collection and detailed descriptions of the exploratory data analysis phases, data preprocessing techniques applied to the dataset as per the requirements, prediction model training using the dataset with different machine learning classifiers in the initial stage to classify the fetus's health, next hyperparameter tuning to the best performing selected models to increase the performance

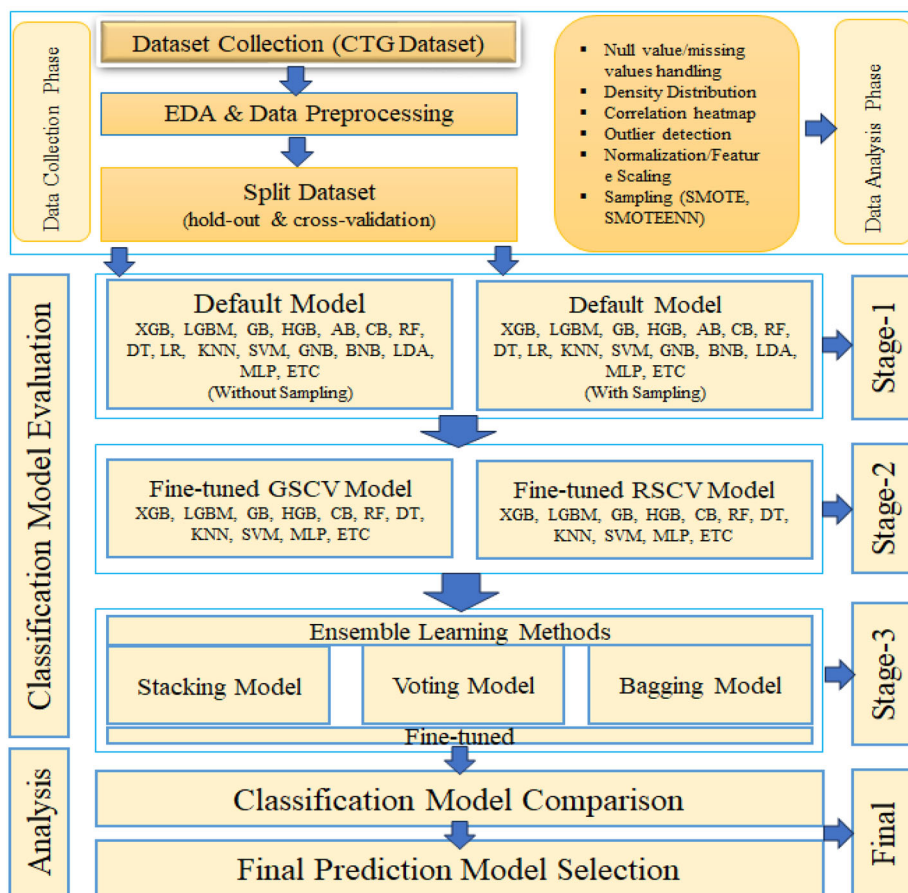


Fig. 1 Work flow diagram of the proposed model

accuracy, and lastly, the ensemble learning approaches like stacking, bagging, and voting methods applied to further increase the diagnosis accuracy of fetal health classification. The mentioned stages are elaborated in detail below accordingly. The work flow diagram of the proposed methodology is depicted in Fig. 1.

3.1 Dataset collection and description

This proposal considered an open-source fetal health classification dataset [30] that is used to classify the health status of the fetus to prevent the child and maternal mortality. The risk factors of fetal health, like fetal movement, uterine contraction, fetal heart rate, and more, were assessed from the conducted experiment through Cardiotocogram (CTG) examination, and the values were recorded. The collected dataset contained 2126 instances with 21 features, along with the one target attribute as ‘fetal health’ extracted from the CTG examination. The outcome, or the target attribute, was classified by three professional obstetricians into three classes as suspect, pathological, and normal of the fetal health state. During the exploratory

data analysis (EDA) phase, we had observed that all the features were in numerical values, and it is abbreviated as per Table 2 according to the feature names. Each feature value density distribution is analyzed, and the statistical values like mean, standard deviation, minimum, and maximum value are tabulated in Table 3 for better understanding the statistical measurement. Also, the target column data distribution was observed as 1655 normal fetuses, 295 fetuses with suspects, and 176 fetuses with pathological conditions. The dataset was highly imbalanced, wherein normal was the majority class and pathological and suspect were the minority class.

3.2 Data preprocessing

Feature engineering, or data preprocessing, is the technique of accumulating raw data and converting it to an understandable and readable format for the computer or machine to learn the data pattern. The main steps in data preprocessing are data cleaning, outlier detection, feature transformation, and balancing the target class. In general, raw datasets obtained from any open source or repositories contain unbalanced target classes, missing or null or implausible zero values, a mix of categorical and numerical data, out-of-bound, and unscaled data. Applying machine learning algorithms to these datasets can result in misleading performances, a tendency to overfit or underfit model training, and biased predictions toward a particular class. Due to those issues, we had applied various preprocessing techniques on the collected raw datasets to solve the above-mentioned shortfalls. The preprocessing techniques used in the dataset are described below according to the needs decided through the EDA phases.

3.2.1 Feature correlation and density distribution

During the data analysis phase, we minutely analyzed each feature of the dataset by counting null values and observed that the dataset had no missing or null values or implausible zero values. So, there was no requirement to apply any imputation techniques. Also, we observed that the dataset had no categorical features, and the target outcome, as well as all the features, had numerical values. So, we have not applied any encoding technique, like label encoding or one-hot encoding, to convert the categorical feature values into numerical values.

We measured the feature correlation heatmap and the data density distribution among all features for the statistical analysis. It is utilized to ascertain the linear correlation among the features and to determine whether one has a higher potential for multicollinearity with the target attribute. This statistical metric evaluates the reliability of the prediction model by verifying the correlated values of each cell. The Pearson correlation coefficient (r) between the two attributes was assessed by the mathematical Eq. (1) below between -1 and 1 .

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \quad (1)$$

Such as, r is the correlation coefficient, x_i indicates the values of the attributes X , $\bar{x}$ denotes mean of the attribute's values X , similarly y_i values of the target class Y and $\bar{y}$ defines the mean of the Y values. The feature correlation matrices among all the features including the target variable are given by the heat map represented in Fig. 2, and the histogram diagram of feature density distribution is depicted in Fig. 3.

Table 2 Extracted feature names from CTG with their abbreviation

Feature name	Abb.	Feature name	Abb.
baseline value	F1	histogram_width	F12
accelerations	F2	histogram_min	F13
fetal_movement	F3	histogram_max	F14
uterine_contractions	F4	histogram_number_of_peaks	F15
light_decelerations	F5	histogram_number_of_zeroes	F16
severe_decelerations	F6	histogram_mode	F17
prolonged_decelerations	F7	histogram_mean	F18
abnormal_short_term_variability	F8	histogram_median	F19
mean_value_of_short_term_variability	F9	histogram_variance	F20
percentage_of_time_with_abnormal_long_term_variability	F10	histogram_tendency	F21
mean_value_of_long_term_variability	F11	fetal_health (Target)	-
Data type (numerical): float64			

Table 3 The statistical measurement of each feature values

No	Abb	Mean	Std.	Min.	Max.	Abb.	Mean	Std.	Min.	Max.		
1	F1	133.303857	9.840844	106.0	160.000	F12	70.445908	38.955693	3.0	180.000		
2	F2	0.003178	0.003866	0.0	0.019	F13	93.579492	29.560212	50.0	159.000		
3	F3	0.009481	0.046666	0.0	0.481	F14	164.025400	17.944183	122.0	238.000		
4	F4	0.004366	0.002946	0.0	0.015	F15	4.068203	2.949386	0.0	18.000		
5	F5	0.001889	0.002960	0.0	0.015	F16	0.323612	0.706059	0.0	10.000		
6	F6	0.000003	0.000057	0.0	0.001	F17	137.452023	16.381289	60.0	187.000		
7	F7	0.000159	0.000590	0.0	0.005	F18	134.610536	15.593596	73.0	182.000		
8	F8	46.990122	17.192814	12.0	87.000	F19	138.090310	14.466589	77.0	186.000		
9	F9	1.332785	0.883241	0.2	7.000	F20	18.808090	28.977636	0.0	269.000		
10	F10	9.846660	18.396880	0.0	91.000	F21	0.320320	0.610829	-1.0	1.000		
11	F11	8.187629	5.628247	0.0	50.700	Fetal_health (Target)					1.0	3.000
Non-Null Count: 2126												
Classes: Normal (1), Suspect (2), Pathological (3)												

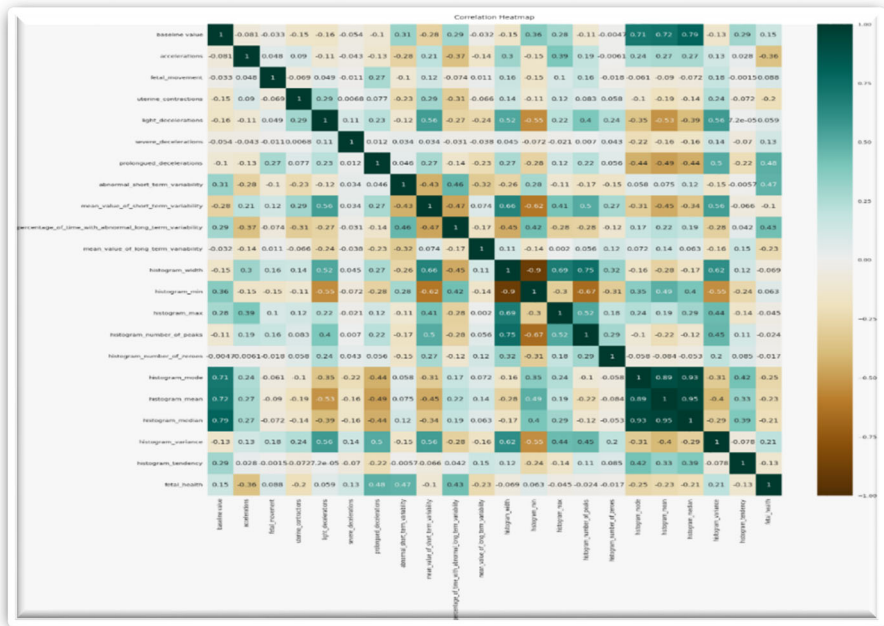


Fig. 2 Feature correlation heat map among all including target attribute

3.2.2 Feature scaling or normalization

Feature scaling is the procedure of normalizing the range of feature values in the dataset by reducing the distance between data points. Scaling the data values makes it easier for a machine learning model to learn and understand the pattern of the data distribution. So, normalization is an essential preprocessing procedure in machine learning that guarantees all feature values possess a comparable scale. This facilitates expedited model training, enhances numerical stability, and mitigates the dominance of features with wider ranges over those with lower ranges. As usual, values in the dataset were in various units and varying scales. Few data had high and low values but not in out of bound and it was detected during the outlier check as per the diagram depicted in Fig. 4. Due to the less outlier values and the uniform distribution of data, we had decided to apply the MinMax scaler normalization technique to scale down the ranges of values in between a fixed range of 0 and 1. The scaled data valued was calculated by using the given formula, such as X is the original value, $X_{\max}$ and $X_{\min}$ are the maximum and the minimum value of corresponding feature, and X' produces the scaled value is represented in formula (2).

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (2)$$

3.2.3 Data balancing

The dataset was highly imbalanced, like the outcome class had an uneven distribution of observations, i.e., some class labels had a very high number of observations and others had a

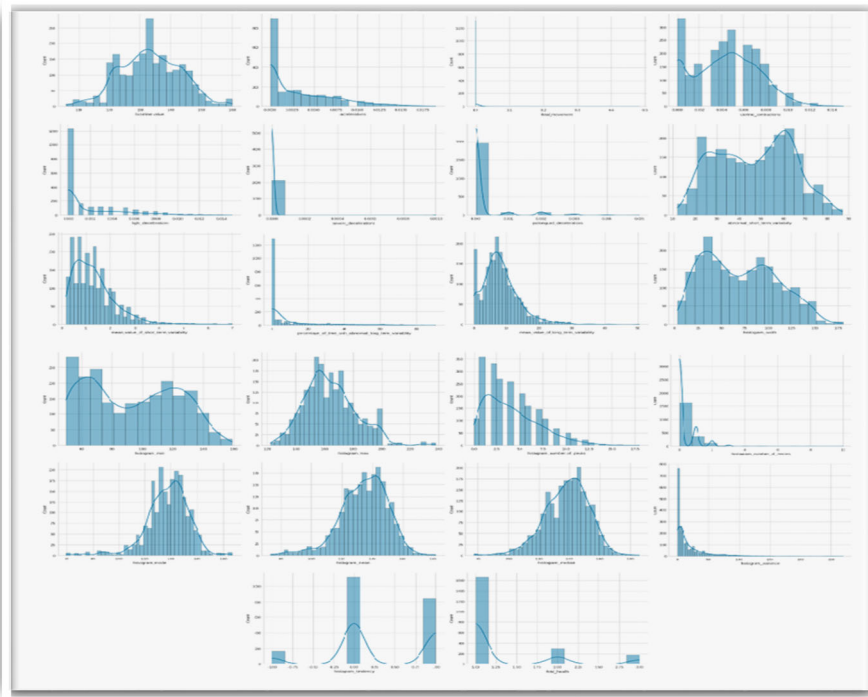


Fig. 3 Feature density distribution

very low number of observations. In datasets like this where the records of a certain class were much more than the other class, the classifier might get biased toward the prediction. Due to the multiclass target values, it is essential to address the data imbalance problem using the sampling techniques. It is highly undesirable that the majority class should be under sampled; better to apply the oversampling technique to increase the minority class with the equal instances of the majority class or to combine the undersampling and oversampling techniques together. The raw dataset had 1655 normal, 295 suspect, and 165 pathological classes. We had initially decided to apply oversampling data balancing techniques, Synthetic Minority Oversampling Technique, or SMOTE, to solve the above problem to avoid biased prediction toward a class. The final dataset had 1655 of each normal, suspect, and pathological class, which adds up to a total of 4965 instances.

The syntactic data sample is created using the following mathematical calculation. Let us, x_i be a minority sample, and the nearest chosen neighbor is x_{ik} . If $x_{ik1}, x_{ik2}, \dots, x_{ikn}$ be the m neighbors, the syntactic values x_{SMOTE} will be generated by the below formula (3).

$$x_{\text{SMOTE}} = x_i + \delta * (x_{ik} - x_i) \quad (3)$$

where δ is a random number within the range of $[0, 1]$, determining the position of the synthetic sample along the line segment connecting x_i and x_{ik} . The process will be repeated until the desired class balance is obtained.

The recently developed enhanced sampling technique SMOTEENN, a version of SMOTE, integrates SMOTE with Edited Nearest Neighbor (ENN), which operates on the fundamental

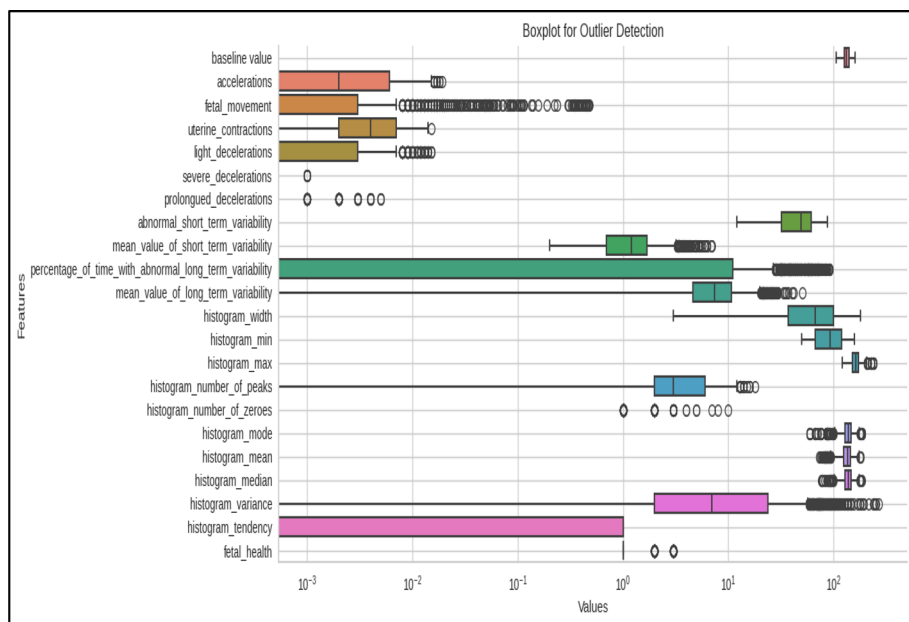


Fig. 4 Boxplot for outlier detection of each feature

principle of KNN, to address target class data imbalance. SMOTE typically generates synthetic samples that are noisy and extraneous in feature space areas where the minority class is densely concentrated. This could result in overfitting and a reduction in the model's generalization ability. The ENN methodology, possessing data filtering capabilities that remove noisy and irrelevant samples from the dataset, can be integrated to address this deficiency. SMOTE generates synthetic instances for the minority class, whereas ENN eliminates specific observations from all classes if it identifies that an observation's class diverges from that of its K -nearest neighbors' majority class. Consequently, by integrating SMOTE with ENN, SMOTENN can generate artificial samples that are less noisy and more indicative of the minority class. This may improve the model's capacity for generalization and class separation by eliminating samples that are overlapped between classes.

For each sample X in the dataset from class C_k is removed if the following formula (4) follows accordingly.

$$\sum_{i=1}^k I(y_i = C_k) < \frac{k}{2} \quad (4)$$

where y_i is the i the nearest neighbor class, C_k is the class of X with number of neighbors k is three, and the indicator function is denoted by $I(y_i = C_k)$ will be 1 if satisfied otherwise 0.

Finally, we had decided to apply both the sampling techniques SMOTE and SMOTEENN on the dataset separately for the model training and evaluate the performance after the effect of the balancing approaches to address the class imbalance problems in the dataset. Before and after the effect of SMOTE and SMOTEENN, the target count is figured out in Table 4 and is represented in Fig. 5.

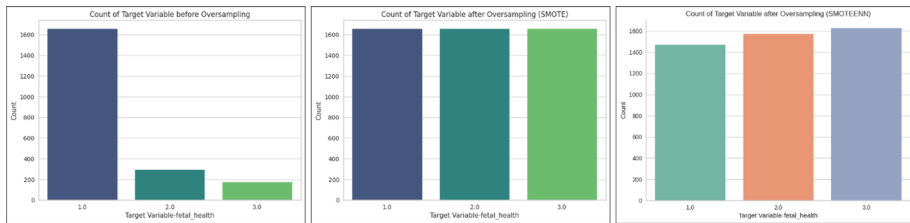


Fig. 5 Before and after applied SMOTE and SMOTEENN, target class data count

Table 4 Target class data distribution before and after sampling technique SMOTE and SMOTEENN

Target Class: fetal_health	Target Instances Count		
	Without Sampling	Sampling-SMOTE	Sampling-SMOTEENN
Normal: 1	1655	1655	1471
Suspect: 2	295	1655	1569
Pathological: 3	176	1655	1625
Total Instances:	2126	4965	4665

Furthermore, label encoding was utilized for the target class, as most machine learning libraries (e.g., Scikit-Learn, TensorFlow, and PyTorch) anticipate class labels to commence at 0, particularly in classification tasks. Numerous algorithms presume labels are indexed from zero; thus, converting (1, 2, 3) to (0, 1, 2) guarantees compatibility.

Also, the entire fetal health prediction model deployment starting from the data collection process including preprocessing methods used and the construction of different model types is represented algorithmically with pseudocode in Algorithm 1.

Algorithm 1: Fetal Health Classification Model on CTG dataset

Input:

- Dataset $D = \{(X_i, y_i) \mid X_i \in \mathbb{R}^m, y_i \in \{1, 2, 3\}, i = 1, 2, \dots, n\}$,
(1: Normal, 2: Suspect, 3: Pathological)
- X_i is the feature vector (fetal heart rate, accelerations, decelerations, etc.): 21
- Models: $M = \{M_1, M_2, \dots, M_{16}\}$
- k-fold Cross-Validation ($k = 10$)
- Hold-out test set
- Hyperparameter space = Θ
- Performance metrics: $M = \{Ac, Pr, Re, Fs, Sd, Cks, Ra, Ma\}$

Output:

- Predicted class labels ($\hat{y}$)
- Performance metrics (Accuracy, Precision, Recall, F1-Score, ROC AUC, Cohen's Kappa, Standard deviation, Mean accuracy)

Stage 0: Data Preprocessing

1. Load CTG dataset
2. Check for null values and handle missing data if any
 $x_{ij} \leftarrow \frac{1}{n_j} \sum_{i=1}^{n_j} x_{ij}$, if $x_{ij} = \text{NaN}$
3. Detect and handle outliers
For each feature x_j , compute Z score:
 $Z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$, Outlier condition: $|Z_{ij}| > \tau = 3$
4. Normalize features using MinMax Scaler
5. Split dataset into training and testing sets
 $(X_{train}, y_{train}), (X_{test}, y_{test}) \leftarrow \text{train_test_split}(X, y)$
6. Apply data sampling methods:
Case: 1 Apply SMOTE to generate synthetic samples:
 $x_{new} = x_{sample} + \lambda(x_{neighbor} - x_{sample})$, $\lambda \sim U(0, 1)$
Case: 2 Apply SMOTEENN
Apply Edited Nearest Neighbors (ENN) to remove noisy samples:
For each instance x_i
Remove x_i if $y_i \neq \text{majority class among } k \text{ nearest neighbors}$

Stage 1: Default machine learning model deployment

7. Define a set of Sixteen ML models (e.g., SVM, DT, RF, XGB, ETC, MLP, etc.)
8. Train each model on:
 - a. Preprocessed dataset (without sampling)
 - b. SMOTE-applied dataset
 - c. SMOTEENN-applied dataset
$$M_i \leftarrow \text{Train}(X_{train}, y_{train})$$
9. Evaluate models using y_{test} , and compute performance metrics ($Ac, Ma, Pr, Re, Fs, Sd, Cks, Ra$)
10. Select the top Eleven best-performing models based on evaluation results (Ma)

Stage 2: Hyperparameter tuning models after sampling SMOTEENN

11. Apply *RandomizedSearchCV* for optimizing hyperparameter tuning on 11 selected models
 $\theta^* = \text{argmaxScore}(M_i(X, y))$, $\theta \in \Theta$
12. Apply *GridSearchCV* for fine-tuning the best parameters on 11 selected models
 $\theta^* = \text{argmaxScore}(M_i(X, y))$, $\theta \in \Theta'$
13. Re-evaluate the models using tuned hyperparameters in both the cases for model selection
 $M_i^* \leftarrow \text{Train}(X_{train}, y_{train}, \theta^*)$
14. Select the best models for constructing the ensemble learning approaches

Stage 3: Ensemble Learning Methods

15. Implement ensemble methods:
 - a. Voting Classifier (Soft Voting) considering different combination of base estimators

$$y_{final} = \text{argmax}_c \sum_{i=1}^N w_i \cdot P(y = c \mid x, M_i)$$

- b. Stacking Classifier with base and meta-learner considering different combinations

$$\begin{aligned} &\text{Train meta-learner } M_{meta} \text{ on model outputs:} \\ &Z = \{M_1(X_{train}), M_2(X_{train}), \dots, M_k(X_{train})\} \\ &M_{meta} \leftarrow \text{Train}(Z, y_{train}) \end{aligned}$$

- c. Bagging Classifier (Bagged Decision Trees, RF, MLP, ETC, HGB, etc.)

For each selected T base models:

$$M_{bagged}(x) = \frac{1}{T} \sum_{t=1}^T M_t(x)$$

16. Compare ensemble model performance using the same evaluation metrics and select best model

$$M^* = \text{argmax}_{M_i \in \{M_{voting}, M_{stacking}, M_{bagging}\}} \max(Ma, Sd, Re)$$

17. Select the best-performing ensemble approach with k-fold mean accuracy

Stage End: Final Model Deployment

18. Train the final selected models on the full training dataset
19. Evaluate on the test dataset
20. Generate a performance report
 $\text{Report} \leftarrow \{M, \text{Confusion Matrix}\}$

End

4 Proposed methodology: prediction models implementation

After applying the requisite data preprocessing approaches on the collected dataset, now the data are ready for the initial machine learning model deployment to classify the fetal health conditions. This research study has decided to initially consider sixteen different machine learning classifiers for predicting fetal health status. The selected classifiers are categorized into boosting-based models: HGB, LGBM, CB, XGB, AB, and GB; tree-based nonlinear models: DT, RF, ETC, KNN, SVM; linear-based models: LDA, LR; probabilistic classifiers: GNB, BNB; and neural network-based classifiers: MLP used to train the dataset for the diagnosis of fetal health risk initially.

4.1 Validation methods

This study employs two distinct model validation approaches: hold-out validation and k -fold validation methods. In the hold-out approach, the entire dataset was split into two parts: 90% of the data instances were used for training the varying models, and the remaining 10% were used in each case to evaluate the multiclass prediction model performance. In the k -fold validation procedure, the dataset is partitioned into k subsets. The classification performance is assessed across k iterations, utilizing $k-1$ groups for model training and one group for model testing. In each iteration, a distinct group is chosen for testing, while the other groups are utilized for training. The classifier's performance is determined by averaging the prediction results across k iterations. To evaluate the mean accuracy of the models, a ten-fold cross-validation method was applied to the dataset.

4.2 Performance metrics

To access the performance of the fetal health prediction models, it is essential to choose the various evaluation metrics for their effectiveness. Through the derived confusion matrix, we can measure the various performance metrics. The several metrics were considered to analyze the performance of the model, namely k -fold mean accuracy (Ma), accuracy (Ac), precision (Pe), recall (Re), Cohen kappa score (Cks), F1-Score (Fs), standard deviation (Sd) of each model, and area under receiver operating characteristics curve (ROC-AUC score) (Ra). The confusion matrix in this context is represented in the following way in Table 5.

In the derived confusion matrix, the True Positives (TP) are the correctly classified instances for each class in the validation dataset. The False Negatives (FN) indicate the misclassified samples that should belong to the actual class but were classified incorrectly. Also, the other two measure the False Positives (FP), which should be the misclassified

Table 5 Derived confusion matrix for the multiclass classification model

Actual class Class names	Predicted class		
	Normal:1	Suspect: 2	Pathological: 3
Normal:0	TP ₁	FN ₁₂	FN ₁₃
Suspect: 1	FN ₂₁	TP ₂	FN ₂₃
Pathological: 2	FN ₃₁	FN ₃₂	TP ₃

Table 6 Calculation of TP, TN, FP, FN for three class classification purpose

Target class	TP	TN	FP	FN
Normal:0	TP ₁	TN ₁ = TP ₂ + TP ₃ + FN ₂₃ + FN ₃₂	FP ₁ = FN ₂₁ + FN ₃₁	FN ₁₂ + FN ₁₃
Suspect: 1	TP ₂	TN ₂ = TP ₁ + TP ₃ + FN ₁₃ + FN ₃₁	FP ₂ = FN ₁₂ + FN ₃₂	FN ₂₁ + FN ₂₃
Pathological: 2	TP ₃	TN ₃ = TP ₁ + TP ₂ + FN ₁₂ + FN ₂₁	FP ₃ = FN ₁₃ + FN ₂₃	FN ₃₁ + FN ₃₂

instances that were incorrectly labeled as a particular class, and True Negatives (TN), which calculate the other correctly classified instances that are not part of the misclassified group.

In this multiclass fetal health classification study, from the confusion matrix, the TP, TN, FP, and FN are calculated by the following formula in each class prediction to derive the accuracy, precision, recall, F1-score, and AUC values of each deployed model. Table 6 is used to calculate the performance measurement of each model by using the mathematical formulas.

The selected five performance measures considered in this study, namely accuracy, precision, recall, F1-score, and AUC value, are calculated using these confusion matrix values. These metrics can be calculated using the formulas (5), (6), (7), and (8) mentioned below:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}} \quad (5)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (6)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (7)$$

$$F1 \text{ score} = 2 * ((\text{precision} * \text{recall}) / (\text{precision} + \text{recall})) \quad (8)$$

This study also examined the widely utilized performance metric, AUC value, which emphasizes the balance between the true-positive rate (TPR) (sensitivity) and the false-positive rate (FPR), rendering it suitable for well-balanced datasets with evenly distributed positive and negative instances, as illustrated in the formulas (9), (10), and (11) below:

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN}) \quad (9)$$

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) \quad (10)$$

$$\text{AUC} = \frac{1}{2} \left(\frac{\text{FP}}{\text{FP} + \text{TN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right), \quad (11)$$

Additional metrics Cohen's Kappa quantifies the concordance between expected and actual classifications, accounting for the agreement that may occur by coincidence in such a way that p_o is the observed agreement proportion of times the classifier agrees with the actual labels and the p_e is the expected agreement by random chance as per formula (12). Finally, the standard deviation and mean accuracy are derived in an expected way from a known method.

$$k = \frac{p_o - p_e}{1 - p_e} \quad (12)$$

4.3 Experimental setup

The initial EDA and the data preprocessing experiment are simulated in the Python 3.7, Jupyter Notebook 6.5.2 environment on an Intel(R) Core (TM) i3-7100U CPU @ 2.40 GHz PC. Also, the high resource requirement is fulfilled by setting the GPU T4 into the Google Colaboratory Compute Engine backend using the Python 3 simulation environment. The normal computation was conducted in the CPU setup, and the high computational requirements for the hyperparameter search of the models using the GridSearchCV and RandomizedSearchCV were fulfilled with an i5 processor, NVidia GeForce GTX GPU, and Windows 10 operating system. Moreover, the ensemble learning methods bagging, voting, and stacking approaches were simulated in a GPU system with a Python programming language environment for better performance.

4.4 Default prediction model and results

Once the necessary steps for preprocessing the data had been taken, the chosen ML models were simulated using the base classifier with the default hyperparameters 'random_state = 0' in each model. Both the hold-out and k-fold validation methods were used to train and test the models. The performance metrics were measured without first balancing the target class data imbalance. Again, after balancing the class imbalance issues by applying the mostly used oversampling method SMOTE and the advanced sampling method SMOTEENN, the experiment was replicated again, and the outcomes of each model's performance were summarized in a good manner. The label encoding was applied on the target class to convert the labels into 0, 1, and 2 due to the models training and testing purpose.

We made a comparative analysis of the performance of all the different ML models wherein it was observed that HGB performed very well with mean accuracies of over 95.92%. But the recall and the standard deviation values were observed very high concerning to learning models due to class imbalance issue and the data pattern. All the parameters for the evaluation of each model are presented in values except the accuracy and the mean accuracy is in percentage. The default hyperparameter tuning model results are presented below in Table 7 before oversampling.

We test the effects of different sampling methods by using the SMOTE oversampling method and the SMOTEENN combined oversampling and undersampling method to deal with an imbalance in the target class. The corresponding experimental results are tabulated in Tables 8 and 9. It is observed that only the nine models were having an accuracy of 95% and above, but the HGB model produces a high mean accuracy of 98.43% with a reduced standard deviation of 0.8801 and a better recall value of 0.99 after the effect of SMOTE.

The simulated results after the effects of SMOTEENN were convenient compared to the other two types of prediction models. Both the HGB and ETC performed well with a mean accuracy of 99.36%, and the standard deviation was observed to be 0.3540, much lower compared to other types of prediction models. The observable recall was 1.00 with a near-zero false negative value for HGB. The confusion matrix of the HGB model after SMOTE and SMOTEENN on the test dataset is depicted in Fig. 6a, b.

Mean accuracy of all the deployed models was considered without applying the sampling default model, and with applying the sampling SMOTE, SMOTEENN performance was compared and depicted in Fig. 7. We observed that the aftereffect of SMOTEENN prediction models produces the best performance concerning mean accuracy, standard deviation, recall values, and other metrics.

Table 7 Default model simulation results before applying sampling methods

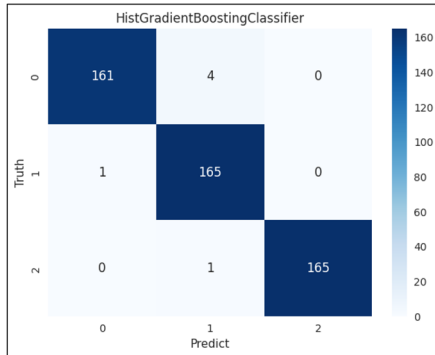
Model	Ac (%)	Ma (%)	Sd	Ra	Pr	Re	Fs	Cks
HGB	95.31	95.92	1.4897	0.98	0.91	0.83	0.87	0.87
RF	94.84	94.51	2.1780	0.99	0.90	0.79	0.84	0.85
XGB	94.84	95.24	1.3664	0.99	0.89	0.83	0.86	0.86
LGBM	94.84	95.51	1.5534	0.98	0.87	0.83	0.85	0.86
CB	94.84	95.19	1.6094	0.98	0.89	0.83	0.86	0.86
GB	93.90	95.04	1.4616	0.98	0.86	0.81	0.84	0.83
ETC	93.43	93.36	2.1996	0.99	0.85	0.74	0.80	0.81
DT	92.02	91.85	2.7909	0.89	0.78	0.77	0.77	0.78
KNN	90.61	91.48	2.0050	0.91	0.78	0.68	0.73	0.73
SVM	88.73	90.59	2.1819	0.96	0.70	0.70	0.70	0.69
AB	88.26	90.49	1.6802	0.94	0.69	0.77	0.73	0.69
MLP	86.85	91.43	1.9899	0.96	0.66	0.70	0.68	0.65
LR	85.92	89.44	1.9245	0.95	0.62	0.66	0.64	0.62
BNB	84.98	83.64	2.3575	0.88	0.59	0.57	0.58	0.59
LDA	84.04	87.82	1.3584	0.94	0.57	0.55	0.56	0.56
GNB	75.59	75.12	4.0590	0.89	0.43	0.81	0.56	0.50

Table 8 Performance analysis of models without hyperparameter tuning with SMOTE method

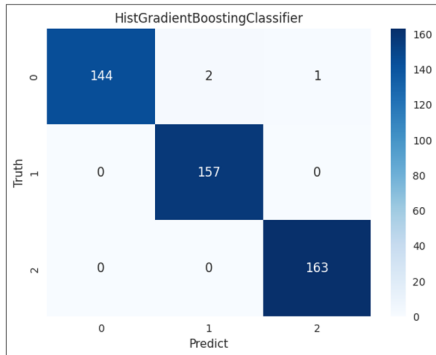
Model	Ac (%)	Ma (%)	Sd	Ra	Pr	Re	Fs	Cks
HGB	98.79	98.43	0.8801	1.00	0.99	0.99	0.99	0.98
RF	98.59	97.85	1.0478	1.00	0.98	0.99	0.99	0.98
ETC	98.39	98.16	1.0362	1.00	0.98	0.99	0.99	0.98
LGBM	98.19	98.21	0.8687	1.00	0.98	0.99	0.98	0.97
XGB	97.99	98.19	0.9699	1.00	0.97	0.99	0.98	0.97
CB	97.99	98.07	0.9359	1.00	0.98	0.99	0.98	0.97
KNN	96.98	96.42	1.0713	0.99	0.96	0.99	0.97	0.95
GB	96.78	96.98	0.8711	1.00	0.96	0.98	0.97	0.95
DT	95.57	95.63	1.2822	0.97	0.95	0.97	0.96	0.93
MLP	94.16	93.44	1.6572	0.99	0.92	0.98	0.95	0.91
SVM	93.56	92.43	1.1945	0.98	0.91	0.97	0.94	0.90
AB	91.35	90.33	2.0367	0.97	0.90	0.96	0.93	0.87
LR	89.54	86.84	1.3478	0.96	0.87	0.91	0.89	0.84
LDA	86.92	85.47	1.4917	0.95	0.83	0.90	0.86	0.80
GNB	75.65	75.69	1.9730	0.89	0.69	0.77	0.72	0.63
BNB	71.83	71.04	2.0205	0.82	0.67	0.71	0.69	0.58

Table 9 Performance analysis of all deployed model after effect of SMOTEENN

Model	Ac (%)	Ma (%)	Sd	Ra	Pr	Re	Fs	Cks
ETC	99.57	99.36	0.3991	1.00	0.99	1.00	1.00	0.99
HGB	99.36	99.36	0.3540	1.00	0.99	1.00	1.00	0.99
CB	99.36	99.17	0.4163	1.00	0.99	1.00	1.00	0.99
KNN	99.14	99.17	0.6050	1.00	0.99	1.00	0.99	0.99
XGB	99.14	99.12	0.3201	1.00	0.99	1.00	0.99	0.99
LGBM	99.14	99.19	0.3230	1.00	0.99	1.00	0.99	0.99
RF	98.93	99.19	0.5240	1.00	0.99	1.00	0.99	0.98
GB	98.07	98.21	0.5560	1.00	0.98	0.99	0.98	0.97
MLP	97.64	96.28	1.1951	1.00	0.98	0.98	0.98	0.96
DT	97.43	96.81	0.9047	0.98	0.98	0.98	0.98	0.96
SVM	95.07	95.31	0.9155	0.99	0.93	0.96	0.95	0.93
AB	92.08	91.71	2.1185	0.98	0.92	0.94	0.93	0.88
LR	88.87	89.66	1.7203	0.97	0.86	0.88	0.87	0.83
LDA	87.15	87.66	1.3467	0.96	0.83	0.88	0.85	0.81
GNB	78.80	79.68	1.8426	0.88	0.74	0.73	0.73	0.68
BNB	70.02	73.18	2.3456	0.82	0.66	0.70	0.68	0.55



(a) The confusion matrix of HGB with SMOTE



(b) The confusion matrix of HGB with SMOTEENN

Fig. 6 Confusion matrix of the deployed HGB model after SMOTE and SMOTEENN

4.5 Fine-tuned prediction model and results

Parameter tuning is a method of selecting the optimal hyperparameters for a machine learning model to learn and get trained on the data pattern. Herein the model's parameters are set before the training begins to optimize the initial setting hyperparameters and control aspects like complexity, learning rate, or regularization. In this research, we had decided to use fine-tuning methods like GridSearchCV and RandomizedSearchCV to obtain the best parameters

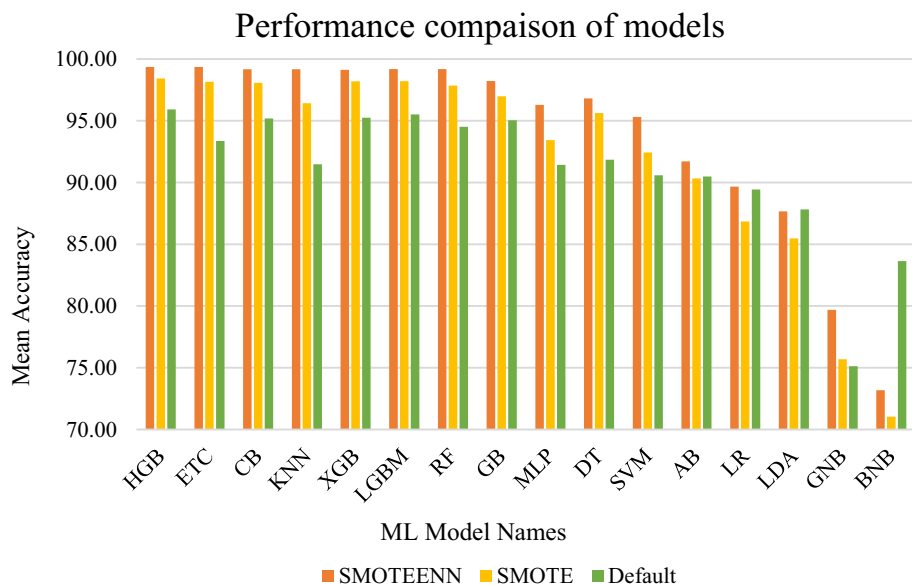


Fig. 7 Performance comparison of SMOTE, SMOTEENN, and default model's mean accuracy

for each ML algorithm to train the dataset. These two strategies are decided due to the medium dataset size and the computational resources required to improve the model performance.

GridSearchCV (GSCV) uses varying combinations of every predefined hyperparameter and their values and calculates the performance for each combination and chooses the best value for the hyperparameters. It is time-consuming and selects the best combination of parameters based on cross-validation performance. On the other hand, RandomizedSearchCV (RSCV) is a method that chooses random combinations of hyperparameters and trains the model to find the best performing set of parameters instead of all combinations. According to Table 9 and Fig. 7, it was observed that the SMOTEENN effect reflected the best results, and we had decided to select the top eleven models with an accuracy of 95% and above. After the application of both the tuning methods on the best performing models, we had extracted the best parameters in each model and deployed these models by passing the optimal hyperparameters with the base classifiers. The best parameters of each machine learning classifier concerning both the tuning method are tabulated in Tables 10 and 11.

We had deployed the models again by considering each model's best parameters along with their estimator value, and the simulation results are summarized in Tables 12 and 13. It was observed that the KNN model performs better in both cases; the mean accuracy is noted as 99.69% and 99.64% concerning RSCV and GSCV. Most of the models performed well with respect to RSCV, but four models, like MLP, ETC, CB, and XGB, showed better in GSCV. So RSCV experimental results were very promising in increasing the mean accuracies.

After the tuning, we got a total of ten models that gave mean accuracies of over 98%, except the DT model result. The hyperparameter-tuned KNN model's confusion matrix in both cases is represented in Fig. 8, and the tuning model's mean accuracy comparison is presented below in Fig. 9.

Table 10 Best parameter with RSCV of different top-performing model

Model	Best Parameters (Randomized Search CV)
HGB	warm_start = False, validation_fraction = 0.1, n_iter_no_change = 10, min_samples_leaf = 30, max_leaf_nodes = 100, max_iter = 200, max_depth = None, max_bins = 255, loss = 'log_loss', learning_rate = 0.2, l2_regularization = 0.0, early_stopping = False
LGBM	Subsample = 1.0, reg_lambda = 0.0, reg_alpha = 0.1, num_leaves = 31, n_estimators = 200, max_depth = -1, learning_rate = 0.1, colsample_bytree = 0.8, boosting_type = 'gbdt'
XGB	Subsample = 0.8, n_estimators = 200, min_child_weight = 3, max_depth = 6, learning_rate = 0.1, colsample_bytree = 1.0
RF	n_estimators = 300, min_samples_split = 2, min_samples_leaf = 1, max_depth = 20, criterion = 'gini'
DT	min_samples_split = 2, min_samples_leaf = 1, max_depth = 20, criterion = 'gini'
SVM	Kernel = 'rbf', gamma = 1, C = 1000, probability = True
KNN	weights = 'distance', n_neighbors = 2, metric = 'euclidean'
CB	n_estimators = 300, max_depth = 10, learning_rate = 0.1
GB	Subsample = 1.0, n_estimators = 300, min_samples_split = 5, min_samples_leaf = 5, max_depth = 10, learning_rate = 0.1
ETC	n_estimators = 300, criterion = 'gini', min_samples_split = 2, min_samples_leaf = 1, max_depth = 20
MLP	solver = 'adam', learning_rate_init = 0.001, hidden_layer_sizes = (128, 64), alpha = 0.001, activation = 'relu'

Table 11 Best parameter with GSCV of different top-performing model

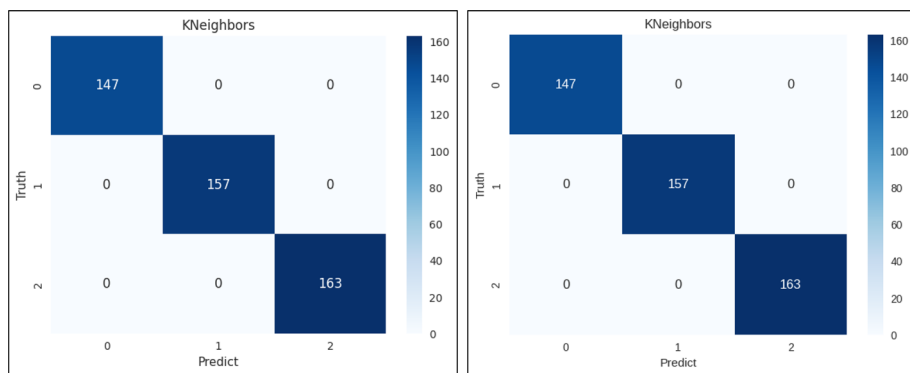
Model	Best Parameters (Grid Search CV)
HGB	learning_rate = 0.4, max_iter = 150, random_state = 0
LGBM	learning_rate = 0.2, random_state = 0, n_estimators = 150
XGB	random_state = 0, eval_metric = 'error', learning_rate = 0.5
RF	criterion = 'gini', n_estimators = 100, random_state = 0
DT	criterion = 'entropy', random_state = 0, max_depth = 12
SVM	Probability = True, C = 1000, kernel = 'rbf', gamma = 1
KNN	n_neighbors = 2
CB	n_estimators = 300, max_depth = 7
GB	warm_start = False, validation_fraction = 0.1, tol = 0.0001, subsample = 1.0, criterion = 'friedman_mse', learning_rate = 0.1, max_depth = 3, n_iter_no_change = None, n_estimators = 100, min_weight_fraction_leaf = 0.0, min_samples_split = 2, min_samples_leaf = 1, min_impurity_decrease = 0.0, max_leaf_nodes = None, max_features = None, init = None, ccp_alpha = 0.0
ETC	n_estimators = 100, criterion = 'gini', random_state = 0
MLP	hidden_layer_sizes = (256,), activation = 'relu', solver = 'adam', learning_rate = "constant", learning_rate_init = 0.01, random_state = 0

Table 12 Performance analysis of the RSCV tuned models

Model	Ac	Ma	Sd	Ra	Pr	Re	Fs	Cks
KNN	100.00	99.69	0.3375	1.00	1.00	1.00	1.00	1.00
GB	99.79	99.17	0.3874	1.00	1.00	1.00	1.00	1.00
SVM	99.79	99.19	0.2650	1.00	1.00	1.00	1.00	1.00
LGBM	99.57	99.19	0.3401	1.00	0.99	1.00	1.00	0.99
ETC	99.36	99.38	0.3562	1.00	0.99	1.00	1.00	0.99
HGB	99.14	99.40	0.3574	1.00	0.99	1.00	0.99	0.99
RF	99.14	99.19	0.6042	1.00	0.99	1.00	0.99	0.99
MLP	99.14	98.59	0.6521	1.00	0.99	1.00	0.99	0.99
XGB	98.72	99.12	0.3539	1.00	0.99	0.99	0.99	0.98
CB	98.72	99.17	0.4793	1.00	0.98	1.00	0.99	0.98
DT	97.22	96.88	0.8419	0.98	0.97	0.98	0.98	0.96

Table 13 Performance analysis of the GSCV tuned models

Model	Ac	Ma	Sd	Ra	Pr	Re	Fs	Cks
KNN	100.00	99.64	0.3409	1.00	1.00	1.00	1.00	1.00
SVM	99.79	99.19	0.2650	1.00	1.00	1.00	1.00	1.00
MLP	99.79	98.65	0.9548	1.00	1.00	1.00	1.00	1.00
ETC	99.57	99.42	0.3991	1.00	0.99	1.00	1.00	0.99
CB	99.36	99.09	0.5197	1.00	0.99	1.00	1.00	0.99
HGB	99.14	99.29	0.2817	1.00	0.99	1.00	0.99	0.99
LGBM	99.14	99.29	0.3533	1.00	0.99	1.00	0.99	0.99
XGB	99.14	99.02	0.3756	1.00	0.99	1.00	0.99	0.99
RF	98.93	99.19	0.5240	1.00	0.99	1.00	0.99	0.98
GB	98.07	98.17	0.5438	1.00	0.98	0.99	0.98	0.97
DT	97.43	97.64	0.7271	0.98	0.98	0.98	0.98	0.96

**Fig. 8** Confusion matrix of tuned KNN model with respect to RSCV and GSCV

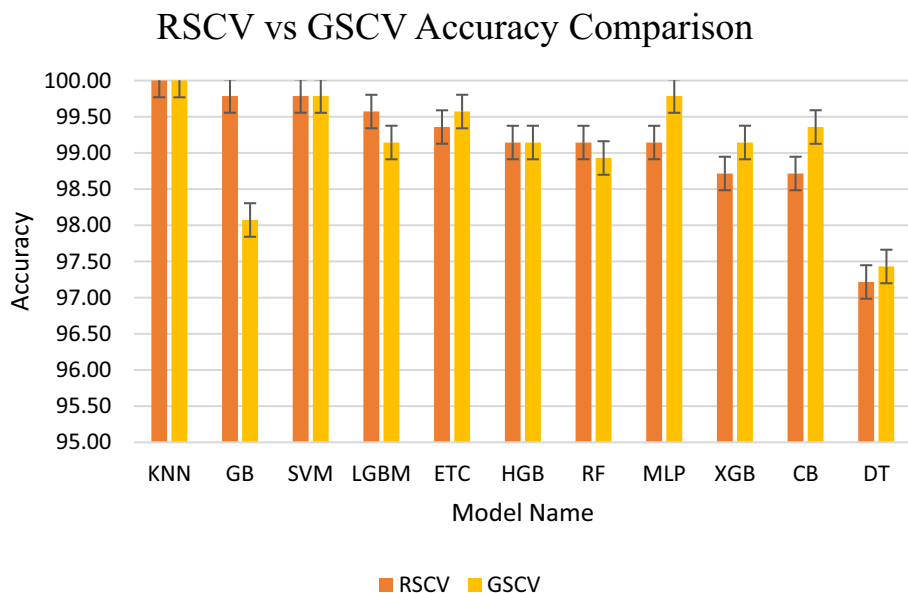


Fig. 9 Performance mean accuracy comparison of all tuned models

4.6 Ensemble learning prediction model

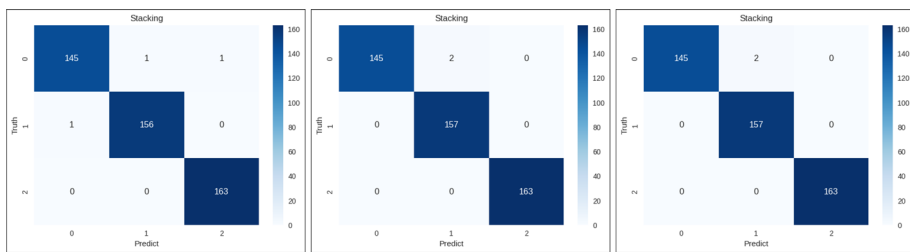
Finally, ensemble learning approaches, including stacking, voting, and bagging, were developed to enhance the predictive performance of the model for mean accuracy and other metrics, hence increasing the model's accuracy and stability by mitigating bias and variation. This can be accomplished only through the utilization of ensembles, which aggregate the insights of numerous models to mitigate bias, variation, or errors, rather than depending on a singular model.

4.6.1 Stacking approach

The stacking ensemble approach employs multiple base models to amalgamate them and yield an ideal predictive model. This is the concept of stacked generalization methodology that integrates several models into a base learner with a final predictive model acting as a meta-learner to enhance forecast accuracy. Unlike averaging or voting methods such as bagging and boosting, stacking optimally integrates various models to enhance predictive accuracy. This study is performed in two stages, stacking by considering the cross-validation method with tenfold. The selected best performing tuned models from RSCV and GSCV were considered as a base learner and the most efficient model as a meta-learner for constructing the stacking classification with hold-out data as well as cross-validation data. The different combination of models was selected, like the top six best performing models, only the boosting-oriented models, and the non-boosting models from fine-tuned were used to perform stacking, which gave a whopping accuracy of 99.60%. The best performing KNN as a meta-learner and the rest as base learners like GB, SVM, LGBM, etc., and MLP were included to build the stacking model. The performance of the many types of stack models by considering all other matrices, including Sd and Ra, is indicated in Table 14. The corresponding confusion matrices are presented in Fig. 10 of the stacking model.

Table 14 Performance analysis of the different types stacking prediction model

Stacking model type	Ac (%)	Ma (%)	Sd	Ra	Pr	Re	Fs	Cks
Boosting Models: Base-Learner (HGB, XGB, CB, GB) Meta-Learner (LGBM)	99.36	99.17	0.23	1.00	0.99	1.00	1.00	0.99
Top-Six Models: Base-learners (GB, SVM, LGBM, ETC, MLP) Meta-learner (KNN)	99.60	99.59	0.31	1.00	0.99	1.00	1.00	0.99
Non-Boosting Mod- els: Base-learners (SVM, ETC, RF, MLP) Meta-learner (KNN)	99.57	99.53	0.37	1.00	0.99	1.00	1.00	0.99

**Fig. 10** Confusion matrix of all types stacking prediction model

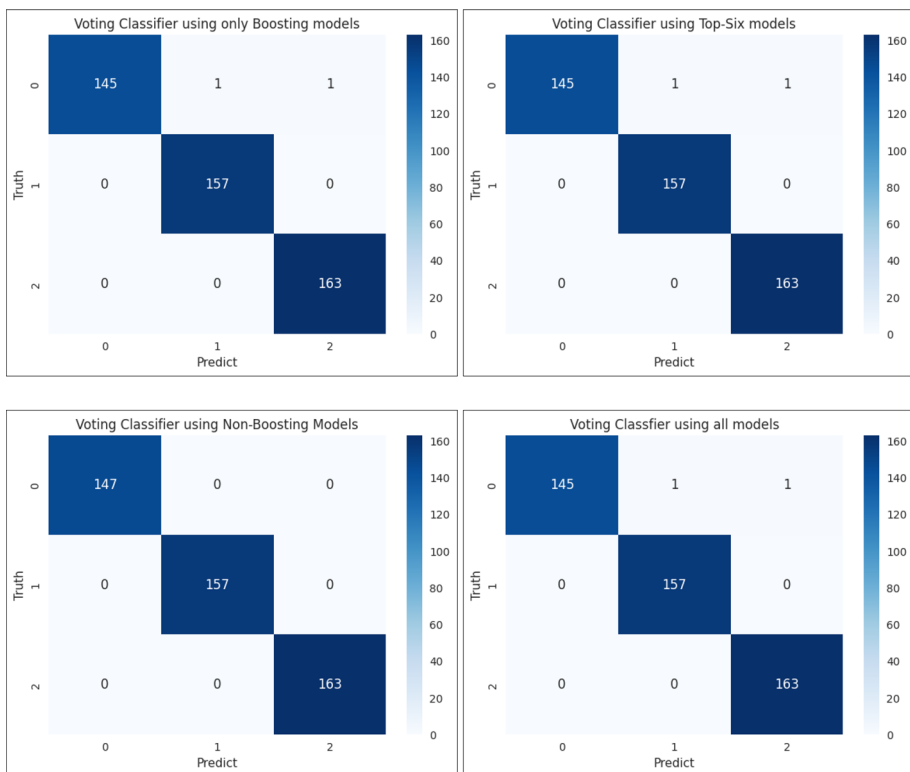
4.6.2 Voting approach

A voting classifier is an ensemble machine learning method that integrates predictions from several independent classifiers, referred to as base classifiers or estimators, to provide a final prediction. It is a model averaging technique in which each base classifier contributes its prediction, with the outcome chosen by a majority vote for classification or an average for regression. The voting classifier is applicable for both binary and multiclass classification applications to improve the accuracy and robustness of the prediction model. In this case, the different combinations among all the best performing models were deployed by considering only boosting and non-boosting classifiers. Also, all non-boosting best performing tuned models that gave a mean accuracy of 99.78% were the best among all the combinations. The results of all voting combinations are presented below in Table 15. The confusion matrices of all the deployed voting models are depicted in Fig. 11 according to the results in Table 15.

The voting ensemble is a superior choice for fetal health categorization compared to the stacking approach, as it decreases variance and enhances generalization. This should function effectively when the base models exhibit diversity in the selection of various learning models. Additionally, it can effectively mitigate the weaknesses of individual models. Certain deficiencies must be rectified, as performance may not improve when worse models are

Table 15 Experimental results of the Voting ensemble classifier model

Voting model type	Ac (%)	Ma (%)	Sd	Ra	Pr	Re	Fs	Cks
Boosting Models: Estimator: LGBM, HGB, XGB, CB, GB	99.57	99.24	0.40	1.00	0.99	1.00	1.00	0.99
Top-Six Models: Estimators: KNN, GB, SVM, LGBM, ETC, MLP	99.77	99.36	0.42	1.00	0.99	1.00	1.00	0.99
Non-Boosting Models: Estimators: SVM, ETC, RF, MLP, KNN	100.0	99.78	0.40	1.00	1.00	1.00	1.00	1.00
Combine all Models Estimators: LGBM, HGB, XGB, CB, GB, SVM, ETC, RF, MLP, KNN	99.57	99.40	0.36	1.00	1.00	1.00	1.00	0.99

**Fig. 11** Confusion matrix of all voting models

incorporated. The voting method is resource-intensive due to numerous substantial models but compared to stacking, it is less computationally expensive. The soft voting approach with probability averaging of each estimator's results concerning every class is considered a voting type.

4.6.3 Bagging approach

Bootstrap aggregating is an ensemble learning method that enhances model performance by diminishing variance and mitigating overfitting. A bagging classifier is an ensemble learning model that utilizes the bagging approach for classification tasks. Bagging is a type of meta-estimator; internally, it partitions the original dataset into several smaller units and applies a machine learning model as an estimator to each subgroup independently, then aggregates predictions from all the models using the voting or averaging ensemble technique to reach the final prediction result. It utilizes large variance models that learn independently from one another in parallel. It is an ensemble meta-algorithm that tries to improve the stability and accuracy of the ML algorithms used in statistical classification and regression. It lowers variance and helps to minimize overfitting. It is frequently utilized in decision tree methods. In this regard, a pipeline is implemented to fit the base estimator and further test the accuracy of previously tuned models. The best performing models from hyperparameter tuning were used as base estimators to build the bagging classifier models. Then bootstrap aggregating, or bagging, was applied to each model to improve accuracy while lowering variance and correcting overfitting. The highest performance with an accuracy of 98.54% and 99.45% of MLP and KNN each was obtained in this ensemble approach. The details result is depicted in Table 16, and the corresponding confusion matrix is represented in Fig. 12.

The bagging classifier method is suitable to reduce overfitting due to the nature of training on different subsets, which can lead to reduced variance. It is helpful to handle noisy data because it is less sensitive to outliers due to the averaging of multiple predictions of subsets and enabling parallel computation. Due to its computationally expensive nature and the scope of improvement in results for already stabilized models. Among the other two approaches, bagging is less effective and poor generalization due to its complex and expensive behaviors.

Table 16 Experimental results of the Bagging ensemble classifier models

Base estimator model	Ac	Ma	Sd	Ra	Pr	Re	Fs	Cks
MLP	99.79	99.72	0.6131	1.00	1.00	1.00	1.00	1.00
KNN	99.69	99.58	0.3471	1.00	1.00	1.00	1.00	0.99
SVM	99.57	99.43	0.2753	1.00	1.00	1.00	1.00	0.99
GB	99.36	99.12	0.3866	0.99	0.99	0.99	0.99	0.99
ETC	99.36	99.12	0.3762	0.98	0.99	0.99	0.99	0.99
LGBM	98.93	98.78	0.3418	0.98	0.99	0.99	0.99	0.98
HGB	98.93	99.10	0.3674	0.99	0.99	0.99	0.99	0.98
XGB	98.93	98.63	0.3540	0.99	0.99	0.99	0.99	0.98
RF	98.72	97.89	0.6142	0.98	0.99	0.99	0.99	0.98
CB	98.72	98.32	0.4893	0.97	0.99	0.99	0.99	0.98
DT	98.50	97.56	0.8619	0.97	0.98	0.98	0.98	0.98

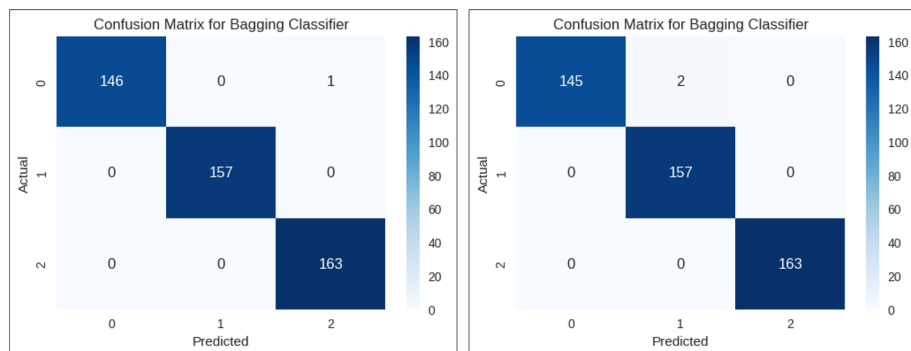


Fig. 12 Confusion matrix of the MLP and KNN bagging ensemble model

5 Comparative results analysis

The comparative results analysis of this study with the prior literary works using the conventional machine learning approaches is presented here. The performances of the fetal health prediction model were evaluated by considering different performance metrics; most of the previously proposed work had claimed their result in terms of accuracy by deploying a few types of algorithms. Here, we considered almost all the algorithms, particularly deploying all boosting, tree-based, linear, probabilistic, and non-boosting models. The comprehensive ensemble learning methods like bagging, stacking, and voting approaches were deployed after tuning the hyperparameters of each model, and their results were calculated by considering almost all the performance matrices like accuracy (Ac), precision (Pr), recall (Re), F1 measure (Fs), ROC-AUC score (Ra), and Cohen kappa score (Cks). Finally, the experimental results were obtained by considering the k -fold CV with mean accuracy and standard deviation of deployed models. In Table 17, the claimed results of the literary using a particular model deployed on the CTG dataset with 21 features and 2126 instances. The accuracy of each literary work is noted down for combative analysis purposes. It is observed that the proposed voting stacking model in this study performs far better than others related to the previous studies with respect to machine learning approaches in terms of mean accuracy.

6 Conclusion, limitations, and future scope

The prime intent of this study was to diagnose the risk of fetal health status through the application of artificial intelligence to reduce the mortality rate of intrapartum, postpartum, and antepartum deaths for the child and the mother in underdeveloped, developing, and low-resource areas. Here, we incorporated sixteen different machine learning models to categorize the risk to fetal health into three classes depending on fetal status. To increase the cross-functionality of the model, we used tenfold cross-validation in every model type to combat overfitting issues and generalization. We used various hyperparameter tuning methods to improve the models' accuracy even more. For example, using GridSearchCV and RandomizedSearchCV helped the models get an accuracy of about 98%, with KNN giving the highest accuracy and k -fold accuracy of 99.69%. Due to the well-managed data preprocessing techniques such as outlier rejection, feature scaling, and the use of the advanced sampling method STOMEENN, the hyperparameter tuning of the models produced a higher

Table 17 Comparative analysis simulated outcomes with the literary works

#	Year	Techniques used	Observable outcomes	Dataset # Instances # Features	Ac (%)
[11]	2015	LR, ANN, SVM, DT (C4.5), CART, KNN, RF, RBFN	Higher accuracy of RF with tenfold CV	CTG [31] # 2126 # 21	99.18
[12]	2020	RF, ANN, CART, REP Tree, C 4.5, SVM, Random Tree, KNN, and bagging ensemble classifier	Higher accuracy of bagging with random forest	CTG [31] # 2126 # 21	99.02
[13]	2022	XGB, LGBM, RF, SVM, KNN, LSTM, ANN	Higher accuracy of RF	CTG [31] # 2126 # 21	98.08
[14]	2021	GB, CB, LGBM, XGB, RF, ET, Cascade Forest Classifier, stacking ensemble	Higher accuracy of blender model soft voting (GB, CB, LGBM, XGB)	CTG [31] # 2126 # 21	95.90
[15]	2018	Decision Forest, Averaged Perceptron, SVM, ANN, LR, Locally-Deep SVM, Boosted Decision Tree, Decision Jungle, Bayes Point Machine	Higher AUC (95.8%), accuracy with Decision Forest model	Clinical # 96 # 22	89.50
[16]	2019	MLP, XGB, DT, RF, LR, SVM Linear Kernel, SVM RBF Kernel, KNN, NB, AB	Higher accuracy with XGB model	CTG [31] # 2126 # 21	93.00
[17]	2021	RF, LR, KNN, GB	Best average accuracy for RF	CTG [31] # 2126 # 21	93.00
[18]	2021	LGBM, RF, GB, ETC, DT, AutoML, Hyperparameter tuning, SHAP	High accuracy with LGBM after hyperparameter tuning with AutoML	CTG [31] # 2126 # 21	95.61
[19]	2021	KNN, SVM, NB, DT (J48), AB, RF, Bagging, and Stacking	Higher accuracy claimed stacking-AB with RF	CTG [31] # 2126 # 21	94.73
[20]	2022	SVM, RF, MLP, KNN	High accuracy achieved by RF with tenfold CV	CTG [31] # 2126 # 21	94.50
[21]	2023	EBBA, GA, qGWO for feature selection, RF, DT, KNN	Combine EBBA and RF achieved high accuracy	CTG [31] # 2126 # 21	96.21

Table 17 (continued)

#	Year	Techniques used	Observable outcomes	Dataset # Instances # Features	Ac (%)
[22]	2023	CB, AB, XGB, LGBM, and GB, ensemble voting,	Soft voting ensemble achieved highest accuracy	CTG [31] # 2126 # 21	99.00
[23]	2024	Ensemble voting methods, LR, RF, GB, and XGB, polynomial expansion	Feature selection with voting (XGB, RF, GB)	CTG [31] # 2126 # 21	99.50
[24]	2024	DNN, LR, KNN, SVM, NB, RF, and GB	Proposed DNN with based model	CTG [31] # 2126 # 21	99.00
[25]	2024	Feature reduction using PCA, Autoencoder, LR, SVM (RBF and Linear), KNN, NB, DT, RF, AB, GB, XGB, and LGBM	PCA with XGB and GB	CTG [31] # 2126 # 21	99.00
[26]	2024	ETSE, SVM LGBM, XGB, KNN, DT, RF, and ETC, voting ensemble, grid search CV	ETSE with voting method after grid search CV	CTG [31] # 2126 # 21	99.66
[27]	2024	DMLP, RF, KNN, XGB, DT, and LR, ANOVA	DLMP with feature selection ANOVA	CTG [31] # 2126 # 21	97.00
[28]	2024	Federated ML with KNN	FML with KNN	CTG [31] # 2126 # 21	99.06
[29]	2025	XGB, query function for the feature selection and used grid search cv for parameter tuning	Active learning (AL) with XGB	CTG [31] # 2126 # 21	99.38
Proposed study of fine-tuned EL methods		Mean accuracy of voting ensemble with fine-tuned SVM, ETC, RF, MLP, and KNN (non-boosting models)		CTG [31] # 2126 # 21	99.78
		Mean accuracy of stacking of fine-tuned with non-Boosting models: Base-learners (SVM, ETC, RF, MLP) and Meta-learner (KNN)			99.60
		Mean accuracy of fine-tuned bagging of MLP model			99.72

mean accuracy. The models performed very well, giving fewer false negative (FN) values and a higher recall value, which is crucial in the diagnosis of fetal health conditions for the decision-making support system, and a moderate standard deviation rate indicates less prone to overfitted models. To take the performance of the model to the next level, we used ensemble learning methods like bagging, stacking, and voting, wherein we observed that the proposed fine-tuned voting, stacking, and bagging method performed very well with an accuracy of 99.78%, 99.60%, and 99.72%, respectively. This study concludes that KNN, HGB, LGBM, MLP, and other boosting algorithms can be very good predictors of fetal health along with the proposed EL methods. The proposed ensemble models classify the dataset with excellent accuracy in identifying whether a CTG record is part of a healthy reading,

a suspected abnormal reading, or a pathological reading. Future studies will benefit from our study by creating more accurate models to distinguish between suspicious and healthy CTG values. The accuracy of the models will increase with repeated training on a larger dataset. Comparing machine learning models and methods can teach clinical specialists a lot and guarantee that the model is applicable to real-world healthcare decision-making support systems. These models make it easier to analyze and interpret data. Our research adds to the body of knowledge on fetal health classification methods by providing a thorough and effective means of enhancing accuracy and interpretability.

Further, we wish to incorporate different feature selection techniques for dimension reduction using LDA, PCA, autoencoders, and clustering into the models with a few important features and deploy a prediction model in future references. Improving model transparency and fostering trust among clinic professionals can be achieved by delving more into explainable AI techniques such as LIME and SHAP. Despite the dataset's small size, all sixteen machine learning models performed well in this investigation. We acknowledge, nonetheless, that larger datasets, including a higher number of CTG reports, could yield a more thorough examination and better model performance. To improve the precision of fetal status predictions, we plan to gather ultrasound data that has been enhanced with more thorough features in the future and build a new model using deep learning techniques. It is possible to create a real-time fetal health system that keeps track of the health of the fetus and sends information to medical professionals. A smart device can accomplish this. Real-time monitoring can greatly improve the quality of prenatal care and the management of high-risk pregnancies.

It is important to recognize the limits of this work. First off, the model has not been evaluated using CTG data in real-time clinical settings, which could affect its applicability and generalizability. Second, the model has not been tested on other datasets because they are not available, which could compromise its robustness and dependability. The possible bias in feature representation is another drawback. Given that the features originate from CTG, the dataset might unintentionally incorporate biases such as gestational age, ethnicity, or maternal health concerns. Lastly, to solve problems in the real world, like adding the model to clinical decision support systems (CDSS), it needs to be rigorously tested and approved by the government. Additionally, real-time fetal monitoring applications may be concerned about latency and computing cost. Future research could entail a thorough validation of the model on patients at local clinics and hospitals to overcome these constraints.

Author contributions The author, SM, conducted the experiments, came up with the concept, wrote the first draft, reviewed and edited it, and formally analyzed the findings. The conceptualization, supervision, investigation, review and editing, validation, and analysis of the findings were overseen by RM and AN. SG reviewed the manuscript and observed the editing and validation. Every writer read and reviewed the manuscript.

Funding To the best of our knowledge, there are no financial or non-financial conflicts of interest for the writers. No entity provided funding for the study.

Data availability The dataset utilized in this research is publicly available in the open-source Mendeley and Kaggle data repository, <https://archive.ics.uci.edu/dataset/193/cardiotocography> and <https://www.kaggle.com/datasets/andrewmvd/fetal-health-classification>. The entire deployment code is in our custody, it will be available whenever it is needed. The first author (Subhash Mondal, ph22cse1001@cit.ac.in) will provide deployment code, and data for further contact upon request.

Declarations

Conflict of interest The authors declare no competing interests.

Ethical approval No human or animal research has been done by any of the authors of this article.

Consent for publication We have no issues with this article being published in this journal.

References

1. Menéndez C, "Barcelona Institute for Global Health (ISGlobal)," (online). Available: <https://www.isglobal.org/en/maternal-child-and-reproductive-health>. (Accessed 15 June 2022)
2. Newborns: improving survival and well-being, 20 September 2020. (online). Available: <https://www.who.int/news-room/fact-sheets/detail/newborns-reducing-mortality>. Accessed 16 June 2022
3. Alipour J, Payandeh A, Karimi A (2023) Prevalence of maternal mortality causes based on ICD-MM: a systematic review and meta-analysis. *BMC Pregnancy Childbirth* 23(1):821
4. Meh C, Sharma A, Ram U, Fu S et al (2022) Trends in maternal mortality in India over two decades in nationally representative surveys. *Int J Obstetr Gynaecol* 129(4):550–561
5. Kodan LR, Verschueren KJC, Roosmalen JV, Kanhai HHH (2017) Maternal mortality audit in Suriname between 2010 and 2014, a reproductive age mortality survey. *BMC Pregnancy Childbirth* 17:275
6. Alfirevic Z, Gyte GML, Cuthbert A, Devane D (2017) Continuous cardiotocography (CTG) as a form of electronic fetal monitoring (EFM) for fetal assessment during labour. *Cochrane Database Syst Rev* 2(2):CD006066
7. Hussain NM, O'Halloran M, McDermott B, Adnan Elahi M (2024) Fetal monitoring technologies for the detection of intrapartum hypoxia—challenges and opportunities. *Biomed Phys Eng Express* 10(2):022002
8. Aeberhard JL, Radan A-P, Soltani RA, Strahm KM (2024) Introducing artificial intelligence in interpretation of foetal cardiotocography: medical dataset curation and preliminary coding—an interdisciplinary project. *Methods Protoc* 7(1):38251198
9. Kinoshita H, Fukunishi H, Shibata C, Hirakawa T (2025) "Neonatal asphyxia prediction using features extracted from cardiotocography data by explainable artificial intelligence. *Inform Med Unlock* 54:101636
10. Miyata K, Shibata C, Fukunishi H, Hemmi K (2025) Cardiotocography-based experimental comparison of artificial intelligence and human judgment in assessing fetal asphyxia during delivery. *Cureus* 17(1):78282
11. Sahin H, Subasi A (2015) Classification of the cardiotocogram data for anticipation of fetal risks using machine learning techniques. *Appl Soft Comput* 33:231–238
12. Subasi A, Kadasa B, Kremic E (2020) Classification of the cardiotocogram data for anticipation of fetal risks using bagging ensemble classifier. *Procedia Comput Sci* 168:34–39
13. Rahmayanti N, Pradani H, Pahlawan M, Vinarti R (2022) Comparison of machine learning algorithms to classify fetal health using cardiotocogram data. *Sixth Inf Syst Int Conf* 197:162–172
14. Li J, Liu X (2021) Fetal health classification based on machine learning. In: *Artificial Intelligence and Internet of Things Engineering (ICBAIE)*, Nanchang, China
15. Akbulut A, Ertugrul E, Topcu V (2018) Fetal health status prediction based on maternal clinical history using machine learning techniques. *Comput Methods Programs Biomed* 163:87–100
16. Hoodbhoy Z, Noman M, Shafique A, Nasim A (2019) Use of machine learning algorithms for prediction of fetal risk using cardiotocographic data. *Int J Appl Basic Med Res* 9(4):226–230
17. Pradhan AK, Rout JK, Maharana AB, Balabantaray BK (2021) A machine learning approach for the prediction of fetal health using CTG. In: *19th OITS international conference on information technology (OCIT)*, Bhubaneswar, India
18. Dwivedi P, Khan AA, Mugde S, Sharma G (2021) Diagnosing the major contributing factors in the classification of the fetal health status using cardiotocography measurements: an AutoML and XAI approach. In: *13th international conference on electronics, computers and artificial intelligence (ECAI)*, Pitesti, Romania
19. Noor NFM, Ahmad N, Noor NM (2021) Fetal health classification using supervised learning approach. In: *National biomedical engineering conference*, Kuala Lumpur
20. Mehbodniya A, Prabhu Lazar AJ, Webber J, Sharma DK (2022) Fetal health classification from cardiotocographic data using machine learning. *Expert Syst Adv Robot Healthc/Intell Softw Eng*. <https://doi.org/10.1111/exsy.12899>
21. Sharma P, Sharma K (2022) Fetal state health monitoring using novel Enhanced Binary Bat Algorithm. *Comput Electr Eng* 101:108035

22. Duhayyim MA, Abbas S, Hejaili AA, Kryvinska N (2023) Ensemble learning for fetal health classification. *Comput Syst Sci Eng* 47(1):823–842
23. Kuzu A, Santur Y (2023) Early diagnosis and classification of fetal health status from a fetal cardiotocography dataset using ensemble learning. *Diagnostics* 13(15):2471
24. Mushtaq G, Veningston K (2024) AI driven interpretable deep learning based fetal health classification. *SLAS Technol* 29(6):100206
25. Dash D, Kumar M (2024) An ensemble-based stage-prediction machine learning approach for classifying fetal disease. *Healthc Anal* 5:100322
26. Talukder MSH, Akter S (2024) An improved ensemble model of hyper parameter tuned ML algorithms for fetal health prediction. *Int J Inf Technol* 16:1831–1840
27. Sirisha U, Naga Srinivasu P, Padmavathi P, Kim S (2024) Dynamic multi-layer perceptron for fetal health classification using cardiotocography data. *Comput Mater Contin* 80(2):2301–2330
28. Nasir MU, Khalil OK, Ateeq K, Almogadwy BS (2024) Federated machine learning based fetal health prediction empowered with bio-signal cardiotocography. *Comput Mater Contin* 78(3):3303–3321
29. Bhardwaj K, Goyal N, Mittal B, Shivhare SN (2025) A novel active learning technique for fetal health classification based on XGBoost classifier. *IEEE Access* 13:9485–9497
30. L. "Fetal Health Classification (online). Available: <https://www.kaggle.com/datasets/andrewmvd/fetal-health-classification>. Accessed 15 Mar 2022.
31. Campos D, Bernardes J (2010) Cardiotocography data set. UCI machine learning repository, 9 June 2010 (online). Available: <https://archive.ics.uci.edu/dataset/193/cardiotocography>. Accessed 18 Mar 2025.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Subhash Mondal (Member, IEEE) received the B.Tech., M.Tech. degrees in Computer Science & Engineering from the University of Calcutta, Kolkata, India, in 2005 and 2007, respectively. He is pursuing a Ph.D. in CSE at the Central Institute of Technology Kokrajhar, Kokrajhar, Assam, India. He is currently working as an Assistant Professor in CSE (AI & ML) at Dayananda Sagar University, Bengaluru, Karnataka, India. He has more than nineteen years of teaching experience. His research interest is in the fields of machine learning, deep learning, and ANN, computer vision, intelligent systems, bio-informatics. He has published more than 37 research publications.



Ranjan Maity (Senior Member, IEEE) received his B. Tech degree from Kalyani University, Kalyani, West Bengal, India, in 2002, his MS degree from Indian Institute of Technology, Kharagpur, India, in 2007, and a Ph.D. degree from Indian Institute of Technology Guwahati, Guwahati, Assam, India, in 2019. He is currently working as an Assistant Professor with the Department of Computer Science and Engineering, Central Institute of Technology Kokrajhar, Assam, India. His research interests include HCI, machine learning, and deep learning. Dr. Maity is also a senior member of IEEE and ACM.



Amitava Nag (Senior Member, IEEE) received his Ph.D. in Engineering from the University of Kalyani, Kalyani, West Bengal, India. He holds an M. Tech in Information Technology from the University of Calcutta, Kolkata, India, in 2005 and a B. Tech from the University of Kalyani, Kalyani, India, in 2003. He is currently working as a Professor of Computer Science and Engineering at the Central Institute of Technology Kokrajhar, Kokrajhar, Assam, India. He has more than 100 research publications in various international journals and conference proceedings. His research interests include IoT, information security, machine learning, deep learning, and NLP. Dr. Nag is a senior Member of IEEE and ACM and a fellow of IET.



Soumadip Ghosh (Member, IEEE) earned his Ph.D. (Engineering) degree from the University of Kalyani, West Bengal, India, in 2017. He received his M. Tech (CSE) and B. Tech (CST) degrees from the University of Calcutta, Kolkata, and the University of Kalyani, Kalyani, WB, India, in 2005 and 2002, respectively. He is presently working as a Professor in the Department of Computer Science and Engineering at Future Institute of Technology, Kolkata, West Bengal, India. He has contributed to numerous research articles in various journals, book chapters, and conferences of repute. Dr. Ghosh is a member of IEEE, IET, and ACM. He has also served as a reviewer for reputed international journals such as ACM, IEEE, Springer, Elsevier, etc.